Register Description for General Purpose Timer Interrupt / PWM

Embedded Controller

Prof. Young-Keun Kim

1. Timer Interrupt (No Output)

Timer Interrupt

● TIMx Setting and TIMx Interrupt (no output mode)

System Clock setting

1. RCC setting (HSI/HSE/PLL)

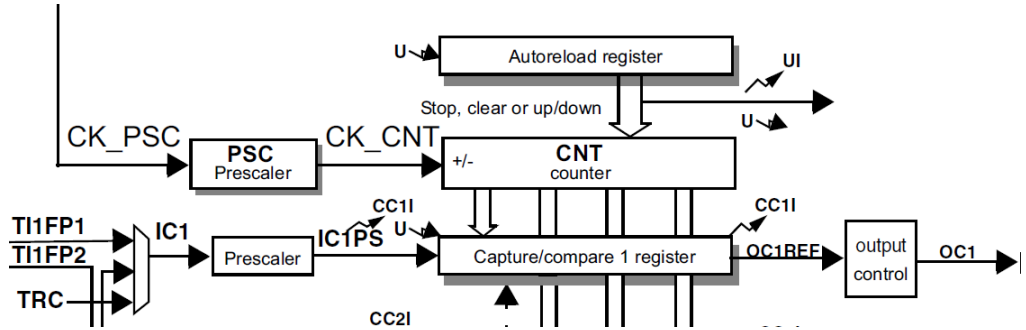
Timer setting

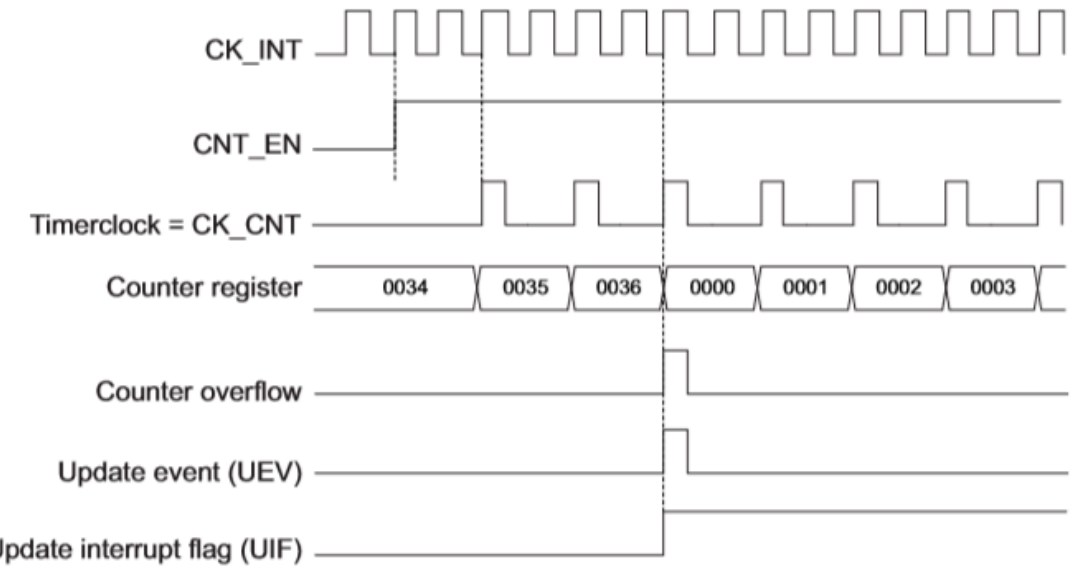
1. Enable Timer Peripheral Clock	(**RCC → APB1ENR**)
2. Set Timer Clock Pre-scaler value	(**TIMx → PSC : PSC[15:0]**)
3. Set Auto-reload value	(**TIMx → ARR : ARR**)
4. Set Counting Direction	(**TIMx → CR1 : DIR**)
5. Enable Update or Compare Interrupt	**(TIMx → DIER : UIE)** or **(: CCyE)**
6. Enable counter	(**TIMx → CR1 : CEN**)

NVIC setting

1. Set interrupt Priority	NVIC_SetPriority(TIMx_IRQn,2)
2. Enable TIMx Interrupt:	NVIC_EnableIRQ(TIMx_IRQn)

*use G.P. Timer (TIM2~) for interrupt
*use S.P. Timer (TIM1) is for PWM output





Timer Interrupt

- **Step 1: Enable timer peripheral clock (APB1ENR)**

Example: TIM2

```
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
```

6.3.11 RCC APB1 peripheral clock enable register (RCC_APB1ENR)

Address offset: 0x40

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PWR EN	Reserved				I2C3 EN	I2C2 EN	I2C1 EN	Reserved			USART2 EN	Reser-ved
			rw					rw	rw	rw				rw	

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	Reserved		WWDG EN	Reserved							TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN
rw	rw			rw								rw	rw	rw	rw

Bit 0 **TIM2EN:** TIM2 clock enable

Set and cleared by software.

0: TIM2 clock disabled

1: TIM2 clock enabled

Timer Interrupt

- **Step 2: Pre-scaler setting (PSC)**
  - Configure counter input clock frequency

Example: TIM2 of 1kHz
- Upcounter Direction
- Set Timer CLK as 84MHz -> 1MHz

  PSC = **83**

13.4.11 TIMx prescaler (TIMx_PSC)

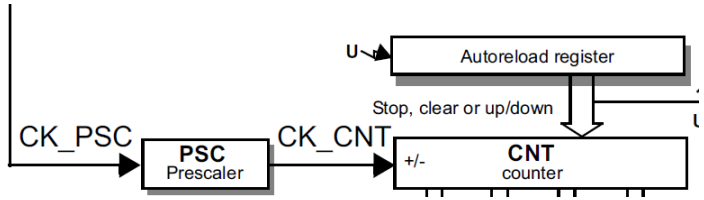
Address offset: 0x28

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency CK_CNT is equal to f_{CK_PSC} / (PSC[15:0] + 1). PSC = **83**
PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in "reset mode").



- f(counter clock) = f(PSC clock) / (PSC + 1)

  = 84MHz / (83 + 1)

  = **1MHz** (주기: 1us)

Timer Interrupt

- **Step 3. Auto-reload setting(ARR)**
  - Configure maximum value of counter

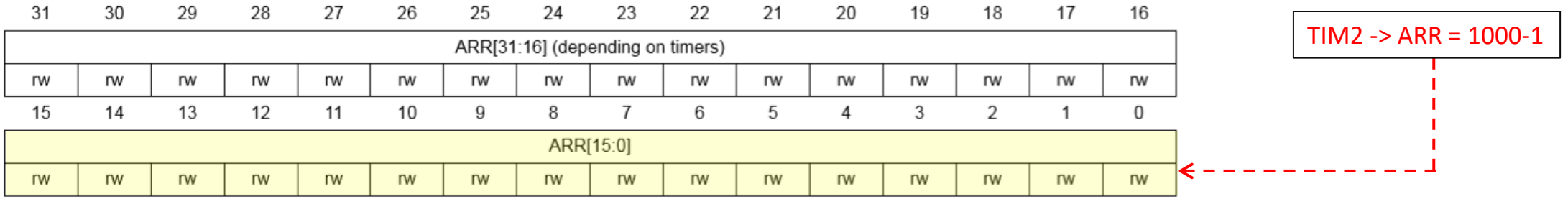
Example: TIM2 of 1kHz
- Set Timer CLK as 84MHz -> 1MHz
- Counter Period of 1 kHz

13.4.12 TIMx auto-reload register (TIMx_ARR)

Address offset: 0x2C

Reset value: 0xFFFF FFFF (for 32-bits timer)
0x0000 FFFF (for 16-bits timer)

f(1 cycle) = f(CK_CNT) / (ARR + 1)
= 1MHz / 1000
= **1 kHz** (주기: 1ms)



Bits 15:0 **ARR[15:0]**: Auto-reload value Counter increases 0 ~ ARR value
ARR is the value to be loaded in the actual auto-reload register.
Refer to the *Section 13.3.1: Time-base unit* for more details about ARR update and behavior.
The counter is blocked while the auto-reload value is null.

Timer Interrupt

- **Step 4, 6: Counting direction setting (CR1)**
  - Set DIR
  - Enable counter

Example: TIM2 of 1kHz
- Up-counter Direction

```
TIM2->CR1      &= ~TIM_CR1_DIR;
                  :
                  :
                  :
TIM2->CR1      |= TIM_CR1_CEN;
```

13.4.1 TIMx control register 1 (TIMx_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 4 **DIR**: Direction

0: Counter used as upcounter — Counter increases 0 ~ ARR value

1: Counter used as downcounter

Note: This bit is read only when the timer is configured in Center-aligned mode or Encoder mode.

Bit 0 **CEN**: Counter enable

0: Counter disabled

1: Counter enabled

Note: External clock, gated mode and encoder mode can work only if the CEN bit has been previously set by software. However trigger mode can set the CEN bit automatically by hardware.

CEN is cleared automatically in one-pulse mode, when an update event occurs.

Timer Interrupt

- **Step 5. Enable Timer Interrupt (DIER)**
  - Enable Update Interrupt
  - Interrupt will be generated when counter resets to 0.

Example: TIM2 of 1kHz
- Upcounter Direction
- Set Timer CLK as 84MHz -> 1MHz
- Counter Period of 1 kHz
- Enable Update Interrupt

```
TIM2->DIER    |= TIM_DIER_UIE;
```

13.4.4 TIMx DMA/Interrupt enable register (TIMx_DIER)

Address offset: 0x0C

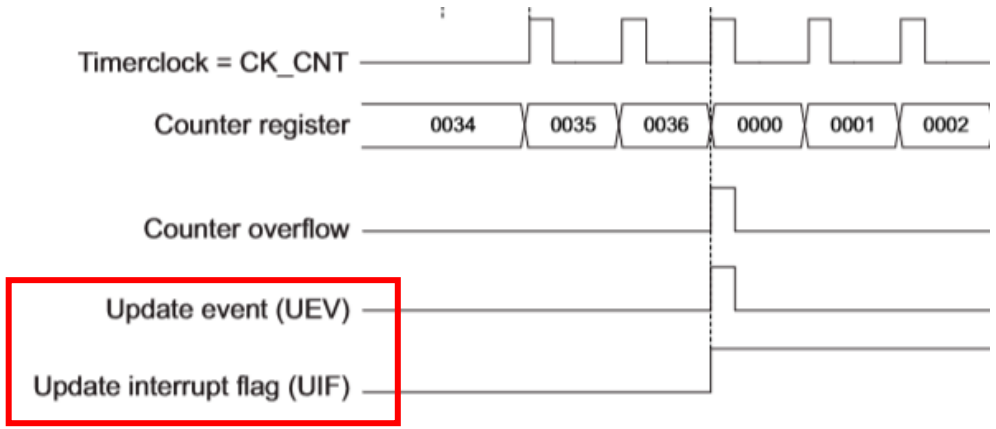
Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TDE	Res	CC4DE	CC3DE	CC2DE	CC1DE	UDE	Res.	TIE	Res	CC4IE	CC3IE	CC2IE	CC1IE	UIE
	rw		rw	rw	rw	rw	rw		rw		rw	rw	rw	rw	rw

Bit 0 **UIE**: Update interrupt enable

0: Update interrupt disabled

1: Update interrupt enabled



Timer Interrupt

- **NVIC Setting: TIMx Interrupt**
  - Enable TIMx IRQ
  - Set priority

TIM2 Interrupt 설정(priority = 2)

```
NVIC_EnableIRQ(TIM2_IRQn);
NVIC_SetPriority(TIM2_IRQn, 2);
```

Example: TIM2 of 1kHz

- Upcounter Direction
- Set Timer CLK as 84MHz -> 1MHz
- Counter Period of 1 kHz
- Enable Update Interrupt
- Enable NVIC (TIM2)

- **Define TIMx_IRQHandler()**

Update Event Interrupt 시 다음 함수 실행

```
void TIM2_IRQHandler(void){
    if ((TIM2->SR & TIM_SR_UIF) != 0) { //UIF pended,
        // your code goes here
        // your code goes here
        // your code goes here
    }
    TIM2->SR &= ~TIM_SR_UIF; // Clear UIF flag
}
```

13.4.5 TIMx status register (TIMx_SR)

Address offset: 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CC4OF	CC3OF	CC2OF	CC1OF	Reserved		TIF	Res	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0			rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

Bit 0 **UIF**: Update interrupt flag

" This bit is set by hardware on an update event. It is cleared by software.

0: No update occurred.

1: Update interrupt pending. This bit is set by hardware when the registers are updated:

Exercise: LED Toggle per *n* second

- **Exercise:** Create a program to toggle an LED in every second. Refer to the following code.

```
int count_ms = 0; //[ms]
int count_s  = 0; //[s]

int main() {
  SysClk_Init();
  LED_Init();
  TIM2_Init();
  NVIC_EnableIRQ(TIM2_IRQn);
  NVIC_SetPriority(TIM2_IRQn, 2);
  while(1);
}

void TIM2_IRQHandler(void){
    if ((TIM2->SR & TIM_SR_UIF) != 0) { //UIF pended;
      // your code goes here
      // your code goes here
    }
    TIM2->SR &= ~TIM_SR_UIF; // Clear UIF flag
  }
```

TIM2 Initialization

// if (is_CCIF(TIMx, CCnum))
// or if (is_UIF (TIMx))

Appendix: TIMx_TypeDef

#include “STM32f411xe.h”

```
/*!< APB2 peripherals */
#define TIM1_BASE (APB2PERIPH_BASE + 0x0000UL)
#define TIM1 ((TIM_TypeDef *) TIM1_BASE)

/*!< APB1 peripherals */
#define TIM2_BASE (APB1PERIPH_BASE + 0x0000UL)
#define TIM2 ((TIM_TypeDef *) TIM2_BASE)
```

```
typedef struct
{
  __IO uint32_t CR1;         /*!< TIM control register 1,              Address offset: 0x00 */
  __IO uint32_t CR2;         /*!< TIM control register 2,              Address offset: 0x04 */
  __IO uint32_t SMCR;        /*!< TIM slave mode control register,     Address offset: 0x08 */
  __IO uint32_t DIER;        /*!< TIM DMA/interrupt enable register,   Address offset: 0x0C */
  __IO uint32_t SR;          /*!< TIM status register,                 Address offset: 0x10 */
  __IO uint32_t EGR;         /*!< TIM event generation register,       Address offset: 0x14 */
  __IO uint32_t CCMR1;       /*!< TIM capture/compare mode register 1, Address offset: 0x18 */
  __IO uint32_t CCMR2;       /*!< TIM capture/compare mode register 2, Address offset: 0x1C */
  __IO uint32_t CCER;        /*!< TIM capture/compare enable register, Address offset: 0x20 */
  __IO uint32_t CNT;         /*!< TIM counter register,                Address offset: 0x24 */
  __IO uint32_t PSC;         /*!< TIM prescaler,                       Address offset: 0x28 */
  __IO uint32_t ARR;         /*!< TIM auto-reload register,            Address offset: 0x2C */
  __IO uint32_t RCR;         /*!< TIM repetition counter register,     Address offset: 0x30 */
  __IO uint32_t CCR1;        /*!< TIM capture/compare register 1,      Address offset: 0x34 */
  __IO uint32_t CCR2;        /*!< TIM capture/compare register 2,      Address offset: 0x38 */
  __IO uint32_t CCR3;        /*!< TIM capture/compare register 3,      Address offset: 0x3C */
  __IO uint32_t CCR4;        /*!< TIM capture/compare register 4,      Address offset: 0x40 */
  __IO uint32_t BDTR;        /*!< TIM break and dead-time register,    Address offset: 0x44 */
  __IO uint32_t DCR;         /*!< TIM DMA control register,            Address offset: 0x48 */
  __IO uint32_t DMAR;        /*!< TIM DMA address for full transfer,   Address offset: 0x4C */
  __IO uint32_t OR;          /*!< TIM option register,                 Address offset: 0x50 */
} TIM_TypeDef;
```

TIM_TypeDef *timer;

2. PWM

PWM: Register

PWM Register Setting

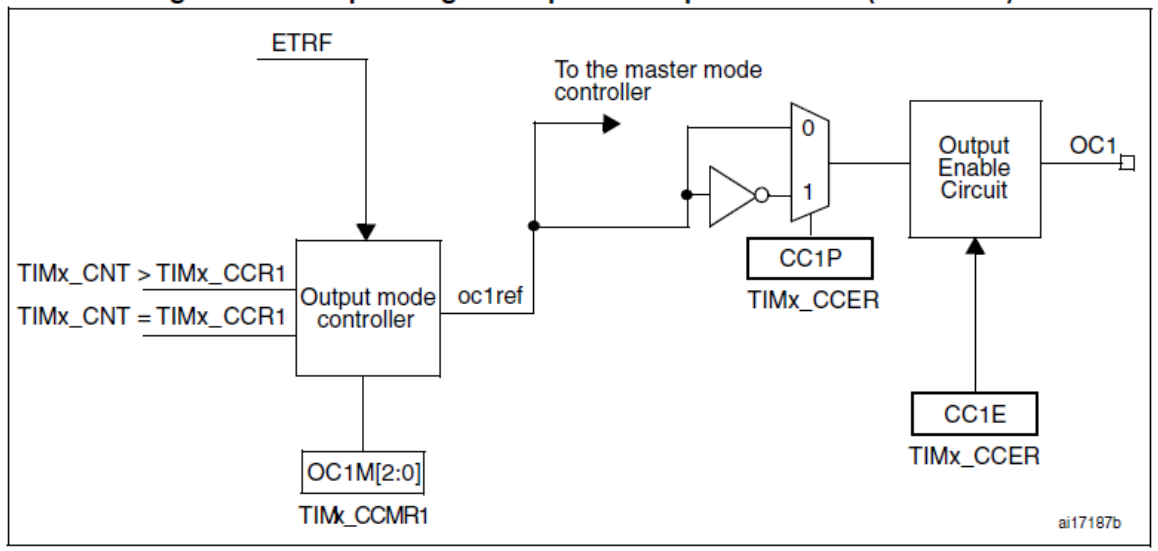
GPIO Pin setting
1. Set RCC
2. AF(TIMx) mode selection for Pin_y in GPIOn

Timer setting (same with interrupt)**:**
1. Enable Timer peripheral Clock (**RCC → APB1ENR**) ***TIM1: RCC_APB2ENR**
2. Set Timer clock Pre-scaler value (**TIMx → PSC : PSC[15:0]**)
3. Set Auto-reload value (**TIMx → ARR : ARR**)
4. Set Counting direction (**TIMx → CR1 : DIR**)
5. Enable Counter (**TIMx → CR1 : CEN**)

*default internal clock source : CK_INT

PWM Output setting:
1. Select Output mode as PWM (**TIMx → CCMR : OCyM**)
2. Set CompareCapture value (**TIMx → CCRy : CCR**)
3. Select Output Polarity (**TIMx → CCER : CCyP**)
4. Enable CompareCaptureOutput (**TIMx → CCER : CCyE**)

Figure 114. Output stage of capture/compare channel (channel 1)



PWM: GPIO_AF Setting

- **GPIO Pin Setting: Alternate Function Output**

  0. Enable GPIO Peripheral Clock (**AHB1ENR**)
  1: Configure as Alternate Function Mode (**GPIOx → MODER**)

EX) TIM2_1 (PA5)
- in GPIOA

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..E and H)

Address offset: 0x00

Reset values:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

```
#define LED_PIN 5

GPIOA->MODER    &= ~(0x03 << (2*LED_PIN));
GPIOA->MODER    |= 0x02 << (2*LED_PIN);
```

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O direction mode.
00: Input (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

PWM: GPIO_AF Setting

- **GPIO Pin Setting: Alternate Function Output**

  2. GPIO alternate function register (AFR)

EX) TIM2_1 (PA5)

- AF1(TIM2) for Pin 5 → AFRL_5

8.4.9 GPIO alternate function low register (GPIOx_AFRL) (x = A..E and H)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRLy:** Alternate function selection for port x bit y (y = 0..7)

These bits are written by software to configure alternate function I/Os

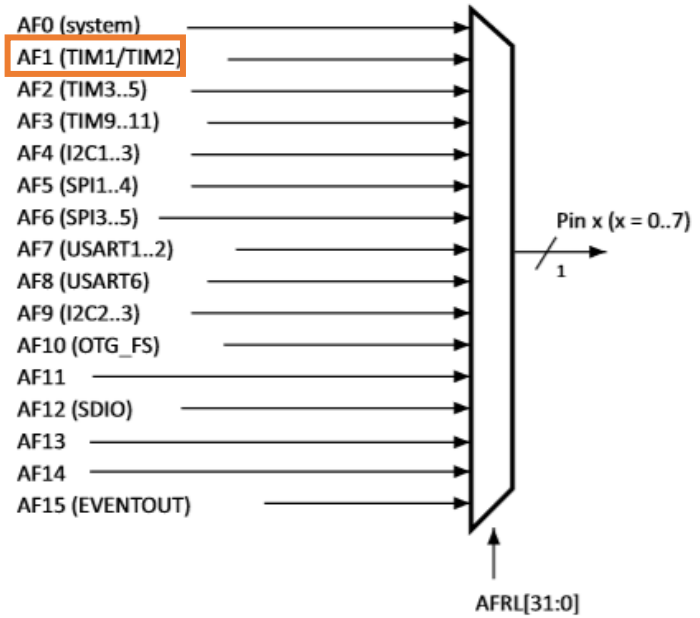
AFRLy selection:

0000: AF0
0001: AF1
0010: AF2
0011: AF3
0100: AF4
0101: AF5
0110: AF6
0111: AF7
1000: AF8
1001: AF9
1010: AF10
1011: AF11
1100: AF12
1101: AF13
1110: AF14
1111: AF15

AFRL: PIN 0~7
AFRH: PIN 8~15

```
GPIOA->AFR[0]   |= 0x01 << (4*LED_PIN);
```

For pins 0 to 7, the GPIOx_AFRL[31:0] register selects the dedicated alternate function



PWM: GPIO_AF Setting

- **GPIO Pin Setting: Alternate Function Output**

  3. Set I/O output speed (OSPEEDR)
     - Set as ‘Very High Speed’

  4: Configure pull-up/down resistors (PUPDR)
     - Set as ‘No-PUPD’

  5: For Output: Configure Output Type (OTYPER)
     - Set as ‘Push-Pull’

PWM: Timer Setting

- **Timer/Counter Setting**

1. Enable Timer peripheral Clock (**RCC → APB1ENR**)
2. Set Timer clock Pre-scaler value (**TIMx → PSC : PSC[15:0]**)
3. Set Auto-reload value (**TIMx → ARR : ARR**)
4. Set Counting direction (**TIMx → CR1 : DIR**)
5. Enable Counter (**TIMx → CR1 : CEN**)

Ex: TIM2_CH1 for PWM output

- Upcounter Direction
- Set Timer CLK as 84MHz -> 1MHz
- Counter Period of 1 kHz
- **No** Update Interrupt

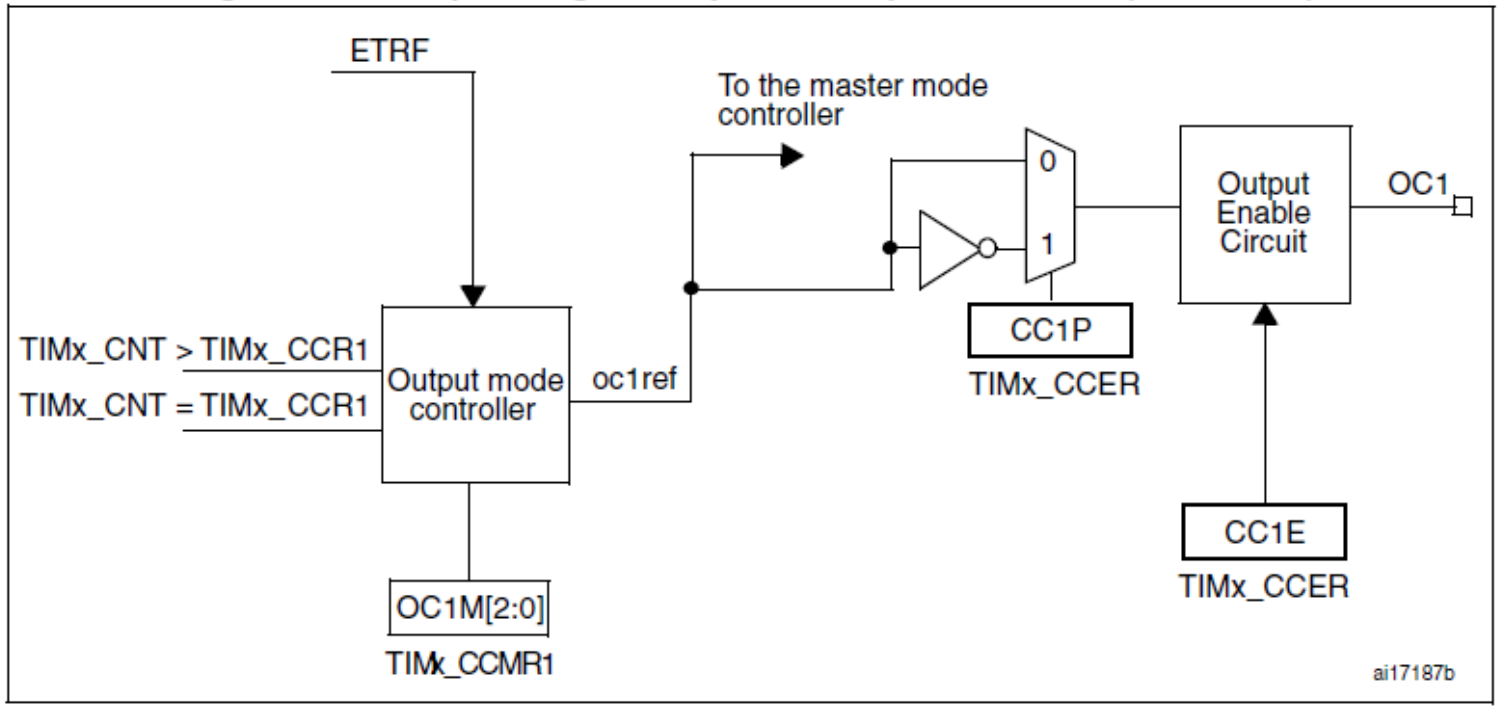
PWM: Register

PWM Register Setting

PWM Output setting:

1. Select Output mode as PWM (**TIMx → CCMR : OCyM**) PWM mode 1 or mode 2
2. Set CompareCapture value (**TIMx → CCRy : CCR**)
3. Select Output Polarity (**TIMx → CCER : CCyP**)
4. Enable CompareCaptureOutput (**TIMx → CCER : CCyE**)
5. *For TIM1 only, enable output **(TIM1 → BDTR)**

Figure 114. Output stage of capture/compare channel (channel 1)



PWM Out Setting

PWM Out Setting

- Step 1: Set PWM Output mode **(TIMx → CCMR : OCyM)**

  (Option): Enable Preload on CCRy **(TIMx → CCMR : OCyPE)**

Ex: TIM2_1A for PWM output

- Upcounter Direction
- PWM period of 1kHz
- Output mode: PWM Mode 1

  TIM2_CCMR->OC1M[2:0]=0110

```
TIM2->CCMR1      &= ~TIM_CCMR1_OC1M;
TIM2->CCMR1      |= TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1M_2;
TIM2->CCMR1      |= TIM_CCMR1_OC1PE;
```

(TIM2_CCMR 1)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2:0]			OC2PE	OC2FE	CC2S[1:0]		OC1CE	OC1M[2:0]			OC1PE	OC1FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Output compare mode

Bits 6:4 **OC1M**: Output Compare 1 mode

110: PWM mode 1

In downcounting, opposite effect.

111: PWM mode 2

Bit 3 **OC1PE**: Output compare 1 preload enable

0: Preload register on TIMx_CCR1 disabled. TIMx_CCR1 can be written at anytime, the new value is taken in account immediately.

1: Preload register on TIMx_CCR1 enabled. Read/Write operations access the preload register. TIMx_CCR1 preload value is loaded in the active register at each update event.

PWM TIM Setting

- **PWM Out Setting**
  - Step 2: Set Compare and Capture value (**TIMx → CCRy : CCR**)

Ex: TIM2_1A for PWM output
- PWM period of 1kHz
- Output mode 2
- 50% Duty ratio
  TIM2->CCR1 = ARR/2

(TIM2_CCR1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CCR1[31:16] (depending on timers)															
rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro

Bits 31:16 **CCR1[31:16]**: High Capture/Compare 1 value (on TIM2 and TIM5). **32-bit**

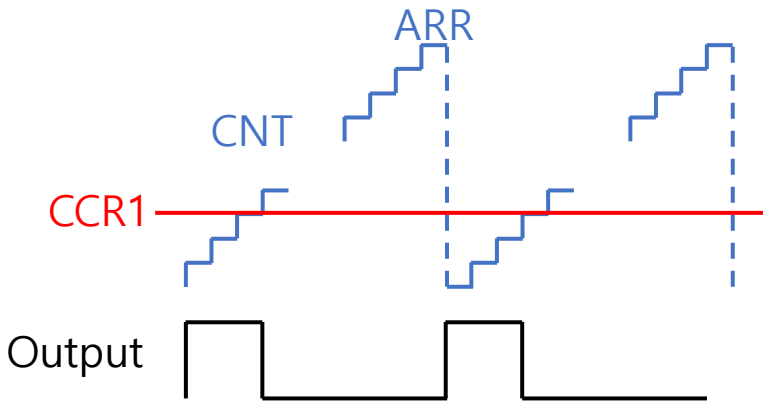
Bits 15:0 **CCR1[15:0]**: Low Capture/Compare 1 value **16-bit**

If channel CC1 is configured as output:

CCR1 is the value to be loaded in the actual capture/compare 1 register (preload value).

It is loaded permanently if the preload feature is not selected in the TIMx_CCMR1 register (bit OC1PE). Else the preload value is copied in the active capture/compare 1 register when an update event occurs.

The active capture/compare register contains the value to be compared to the counter TIMx_CNT and signaled on OC1 output.



PWM TIM Setting

- **PWM Out Setting**
  - Step 3. Select Output Polarity (**TIMx → CCER: CCyP**)
  - Step 4. Enable CompareCaptureOutput (**TIMx → CCER: CCyE**)

Ex: TIM2_1A for PWM output
- PWM period of 1kHz
- Output mode 2
- 50% Duty ratio
  TIM2->CCR1 = (ARR+1)/2

(TIM2_CCER)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	Res.	CC3P	CC3E	CC2NP	Res.	CC2P	CC2E	CC1NP	Res.	CC1P	CC1E
rw		rw	rw	rw		rw	rw	rw		rw	rw	rw		rw	rw

Bit 1 **CC1P**: *Capture/Compare 1 output Polarity.*

CC1 channel configured as output:

0: OC1 active high

1: OC1 active low

```
TIM2->CCER   &= ~TIM_CCER_CC1P;
TIM2->CCER   |=  TIM_CCER_CC1E;
```

Bit 0 **CC1E**: *Capture/Compare 1 output enable.*

CC1 channel configured as output:

0: Off - OC1 is not active

1: On - OC1 signal is output on the corresponding output pin

PWM TIM Setting

- **PWM Out Setting**
  - Step 5: TIM1 Only, enable OC output: (TIMx → BDTR)

```
TIM2->BDTR |= TIM_BDTR_MOE;
```

12.4.18 TIM1 break and dead-time register (TIMx_BDTR)

Address offset: 0x44

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MOE	AOE	BKP	BKE	OSSR	OSSI	LOCK[1:0]		DTG[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 15 **MOE**: Main output enable

This bit is cleared asynchronously by hardware as soon as the break input is active. It is set by software or automatically depending on the AOE bit. It is acting only on the channels which are configured in output.

0: OC and OCN outputs are disabled or forced to idle state.

1: OC and OCN outputs are enabled if their respective enable bits are set (CCxE, CCxNE in TIMx_CCER register).

See OC/OCN enable description for more details (*Section 12.4.9: TIM1 capture/compare enable register (TIMx_CCER)*).

309p

Tutorial: LED Control with PWM

- Exercise: LED brightness should be brighter and darker by controlling PWM duty ratio

```
10  int main() {
11
12
13      Previous Setting for TIM2 PWM
14
15
16      while(1) {
17        for(int i = 0; i < 10; i++){
18          TIM2->CCR1 = i * 1000;
19          delay(100);
20        }
21        for(int i = 0; i < 10; i++){
22          TIM2->CCR1 = (10 - i) * 1000;
23          delay(100);
24        }
25      }
26  }
```